

# Verified-Then-Paid Escrow

*An AI judge whose verdict is fixed on Hedera before anyone sees it, and checkable by anyone afterwards*

**Mohamed Hedi Ben Jemaa** · South Mediterranean University (SMU), Tunis · Technical report, 30 September 2026 · Demo: 3 October 2026

**Abstract.** Freelance platforms and AI graders both decide who gets paid, and neither leaves a record that the losing party can check. Verified-Then-Paid Escrow locks a client's HBAR in a Hedera smart contract, lets a local LLM judge a text deliverable against a Statement of Work, and writes the **complete evaluation record to the Hedera Consensus Service (HCS) before the verdict is revealed** to anyone. Only after the public mirror node confirms those exact bytes does the oracle submit the verdict on-chain, which releases the payment or holds it for a human arbitrator. A public verification page then re-derives everything in the browser, straight from the mirror node: it hashes the app's copy and the anchored copy, checks that the escrow contract commits to the same record, and scans the topic for competing records. A live **psql** attack on the database is caught on stage. The system runs end to end on Hedera testnet: 44 Python tests, 8 contract tests, a four-way cross-language hash check, an 18/18 evaluator suite, and deliverable-to-payment in 47–53 s.

| Hedera services used                                                                                                                                                        | Stack                                                                                                                                                    | Status on 30 Sep                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| HCS (full record anchoring, oracle-only submit key) · Smart contracts (Solidity <b>VerifiedEscrow</b> ) · native HBAR transfers · public mirror node (browser verification) | Next.js 14 · FastAPI · Node/Express + <b>@hashgraph/sdk</b> + <b>ethers</b> v6 · Hardhat (solc 0.8.24) · Postgres 16 · Ollama <b>qwen2.5:7b-instruct</b> | Feature-complete (tag <b>v0.9-feature-complete</b> ); deployed on testnet; 3 seeded demo escrows; all 16 HashScan links resolve |

## 1. Introduction and problem

AI agents are starting to approve work that moves money: reviewing deliverables, signing off QA, grading assignments, settling agent-to-agent tasks. Today, two kinds of trust gap sit underneath these decisions:

**Table 1.** The two problems the project targets (PRD §2).

| Failure mode            | Today                                                               | Consequence                                                             |
|-------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------|
| Centralized arbitration | Platforms decide disputes with no record that anyone else can check | Both parties have to trust the platform                                 |
| Opaque AI judgment      | AI graders produce verdicts nobody can audit afterwards             | An operator can quietly change a verdict and nobody downstream can tell |

What's missing is a **tamper-evident record of what the AI decided, fixed before anyone had the chance to change it**. Our one-line pitch: *the escrow doesn't ask you to trust the AI's verdict — it lets you check it*.

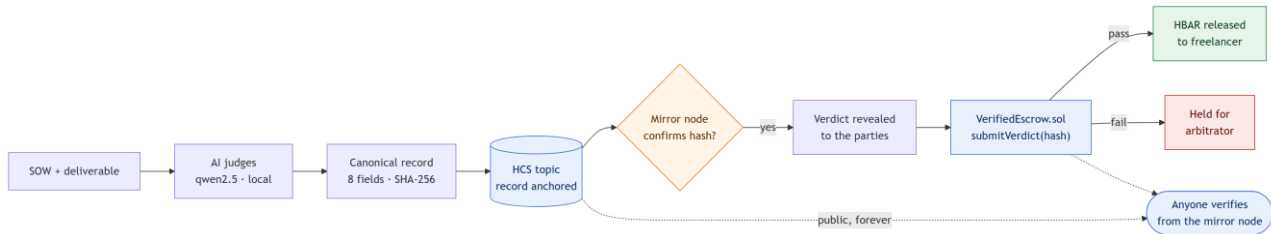
### Goals and scope

- **G1:** a complete flow on Hedera testnet: create a contract, lock HBAR, submit the deliverable, get an AI verdict, anchor it on HCS, then release or hold automatically.
- **G2:** a public verification page that detects any change to the off-chain record by comparing it with the HCS-anchored record.
- **G3:** a rehearsed, reliable stage demo (a 4:10 run sheet) that includes a live **psql** tampering attack.
- **G4:** honest trust claims. The contract trusts an oracle, but the oracle's claims can be publicly checked against HCS.

Deliberately out of scope: real user wallets, file or image deliverables, stablecoins, multi-milestone contracts, and mainnet. Anchoring the full record makes the SOW and the deliverable **public and permanent**, so all demo content is fictional. §6 covers this and the other limitations.

## 2. Solution overview

Four actors share one contract. The **client** (Amira) writes an SOW and funds the escrow. The **freelancer** (Youssef) submits a text deliverable. The **arbitrator** (Nour) resolves held contracts. The **verifier** is anyone, with no login needed. A fifth, system-level actor, the **oracle** (the backend's Hedera account), is the only account allowed to post to the HCS topic and to call `submitVerdict`.



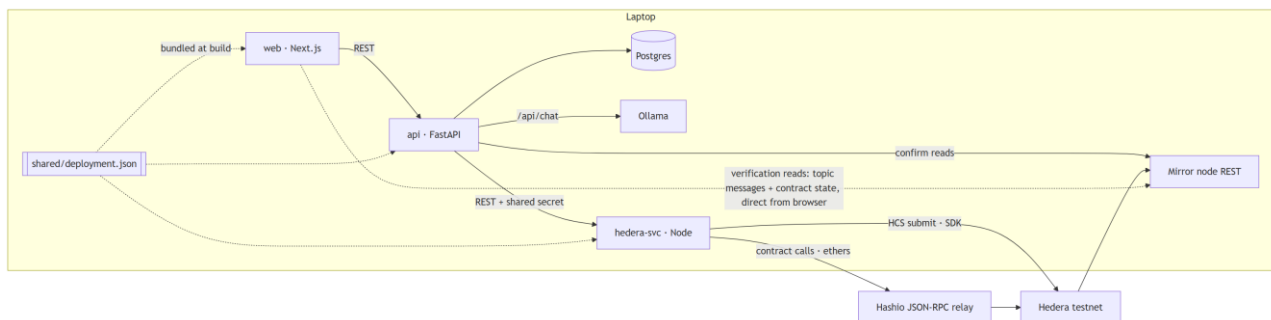
**Figure 1.** The core idea: judge → anchor → confirm → reveal → pay or hold → anyone verifies. The verdict is kept hidden until the mirror node serves the anchored bytes.

The key design choice is the order of these steps. The AI's output is turned into a **canonical record**: SOW, deliverable, verdict, reasoning, model version, escrow ID, schema tag and timestamp. It is hashed and submitted to HCS. The app only reveals the verdict once the mirror node returns bytes whose SHA-256 matches. So by the time anyone sees a verdict, anyone can already verify it, and the operator can no longer change it without being caught. The contract then stores the same hash (`verdictHash`) and the pass/fail flag. This ties the money movement to the anchored record.

**What this does not prove:** it doesn't prove that the model's judgement was correct, or that nobody interfered with the evaluation *before* it was anchored. We state these limitations openly in the UI and in the pitch (§6).

## 3. System architecture

The system is five local processes plus the Hedera testnet. The operator's laptop runs everything except consensus. The only cloud dependency is Hedera itself: the Postgres database is local and the model runs natively through Ollama.



**Figure 2.** Architecture. Solid edges are the write path; dotted edges are verification reads that go from the browser straight to the mirror node, using a topic ID and contract address bundled into the web build.

**Table 2.** Processes and responsibilities (TRD §1).

| Process          | Technology                                        | Role                                                                                              |
|------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------|
| web :3000        | Next.js 14, Tailwind, SWR polling                 | Persona UI and the public <code>/verify</code> page                                               |
| api :8000        | FastAPI, SQLAlchemy 2, httpx                      | Orchestrator: state machine, LLM calls, canonical record, mirror confirmation                     |
| hedera-svc :7000 | Express, <code>@hashgraph/sdk</code> , ethers v6  | Every Hedera write (HCS submits, contract calls). Holds all keys; localhost-only, token-protected |
| db :5432         | Postgres 16 (Docker)                              | Off-chain record store: the thing the demo attacker edits                                         |
| ollama :11434    | <code>qwen2.5:7b-instruct</code> (Q4_K_M), native | Local evaluator, JSON-schema output, temperature 0, seed 42                                       |

**Core rule: the backend relays; it never attests unchallenged.** Every claim the UI shows can be re-derived by the browser from the public mirror node. The browser takes the topic ID and contract address from `shared/deployment.json` at build time, not from `api`. Otherwise a dishonest backend could point the "independent" check at a topic it controls. We found this weakness during the design review, and it is now enforced: if `api` reports a different topic, the page shows a red error.

Some technology choices were made for trust reasons, not convenience. SHA-256 in TypeScript uses `@noble/hashes` rather than `crypto.subtle`, because `crypto.subtle` is undefined outside secure contexts and would crash the verify page on a LAN IP. Contract calls go through `ethers` and the Hashio JSON-RPC relay with ECDSA keys, so `msg.sender` matches the Hardhat deploy path. HCS submits use the Hedera JS SDK.

## 4. Core design

### 4.1 Canonical record and cross-language hashing

Verification only works if Python (which builds the record), Node (which submits it) and the browser (which checks it) produce **byte-identical** serializations. We made that easy to guarantee by restricting the record to eight **string-only** keys: no numbers, booleans, nulls or nesting. Keys are sorted, separators are `,` and `:` with no whitespace, non-ASCII is emitted raw as UTF-8, and the hash is SHA-256.

**Table 3.** The v1 canonical record.

| Key                             | Content                                                                                                            |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>contract_id</code>        | On-chain escrow ID. Binds the record to one escrow, so a passing record can't be replayed against another          |
| <code>sow, deliverable</code>   | Normalized texts, exactly as stored                                                                                |
| <code>verdict, reasoning</code> | <code>pass</code> or <code>fail</code> ; reasoning capped at 3,000 characters in code                              |
| <code>model_version</code>      | <code>ollama/qwen2.5:7b-instruct@845dbda0ea48</code> (tag + digest); <code>replay/...</code> for replayed verdicts |
| <code>schema, timestamp</code>  | <code>vte-record/1</code> ; UTC YYYY-MM-DDTHH:mm:ss.SSSZ                                                           |

**Normalization happens once, in Python, when text is ingested.** It rejects NUL and lone surrogates, converts CRLF to LF, applies NFC, and strips a named set of trailing ASCII whitespace. The review found that Python's `rstrip()` and JavaScript's `trimEnd()` strip *different* characters, so the set is spelled out. Nothing downstream normalizes again. Six shared fixtures (ASCII, French, Arabic, emoji with ZWJ, quotes and backslashes, every control character) are checked four independent ways: pytest, vitest, the browser self-test page, and plain `sha256sum` on the expected bytes. This four-way check was the **Day 1 gate**, and it passed on its first attempt.

### 4.2 HCS anchoring and mirror-node confirmation

Each contract deployment gets its **own topic**. The topic's submit key is the oracle's key, and it has no admin key, so nobody else can post look-alike records and nobody can change the topic's configuration. The message payload is exactly the canonical bytes, with no envelope, so the hash of the reassembled message *is* the record hash. Records are 1.7–4.6 KB, so they are split into 1,024-byte chunks.

- `hedera-svc` checks the hash of the bytes it received before submitting. It uses `executeAll`, because the SDK's `execute()` returns only the first chunk's response. Submissions go through a **single-flight queue**, so chunks from two different records never interleave.
- `api` polls the mirror node, reassembles the chunks by `initial_transaction_id` and chunk number (gaps and foreign chunks are rejected), and marks the record **confirmed** only when SHA-256 of the reassembled bytes equals the record hash.
- Visibility rule:** the verdict, reasoning, criteria results and the `/verify` record are served only once `hcs_anchors.confirmed_at IS NOT NULL`. Even the timeline entry "Evaluation complete" leaves out the verdict.

### 4.3 The VerifiedEscrow smart contract

`VerifiedEscrow.sol` (Solidity 0.8.24, EVM version shanghai) holds each escrow's parties, `sowHash`, amount and status. `createEscrow` is payable and requires three distinct parties. `submitVerdict(id, passed, verdictHash, sowHash)` is callable only by the oracle. It rejects a SOW mismatch or an all-zero hash, stores **both** `verdictHash` and `verdictPassed`, and then either pays the freelancer or marks the escrow `Held`. `resolveDispute` lets only that escrow's arbitrator release or refund a held escrow. Payouts follow checks-effects-interactions.

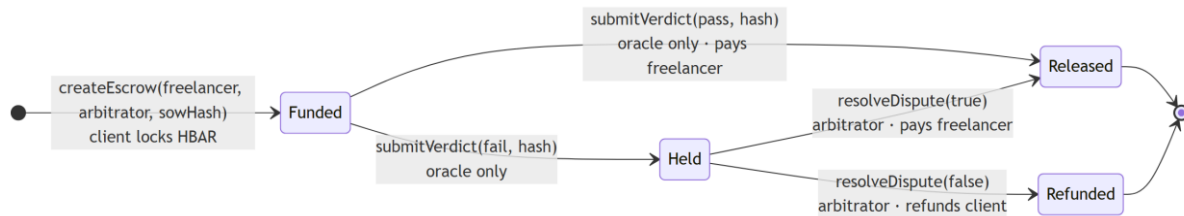


Figure 3. On-chain escrow status lifecycle.

Storing `verdictPassed` is not redundant. Without it, a `Released` status can't tell an oracle pass apart from an arbitrator's release, and the browser couldn't check the second half of the oracle's claim. Hedera's EVM needed four fixes, found and documented during the build:

- **Units:** amounts on-chain are in tinybars ( $10^8$  per HBAR), while `ethers` sends weibars. We always read amounts with `formatUnits(x, 8)`.
- **Hashio relay:** JSON-RPC batching disabled; an explicit legacy `gasPrice` (the EIP-1559 estimate sometimes fell below the relay minimum); a `staticCall` preflight so reverts come back as `409 WOULD_REVERT` with the contract's reason; and pinned-nonce retries so a retried verdict can't apply twice.
- **Nonces:** a per-signer mutex prevents nonce collisions. `gasLimit` is fixed at 400,000, and each call measured **0.0246 ħ**.
- **Accounts:** the four ECDSA accounts are finalized from "hollow" auto-created accounts, and keys are parsed with `PrivateKey.fromStringECDSA`.

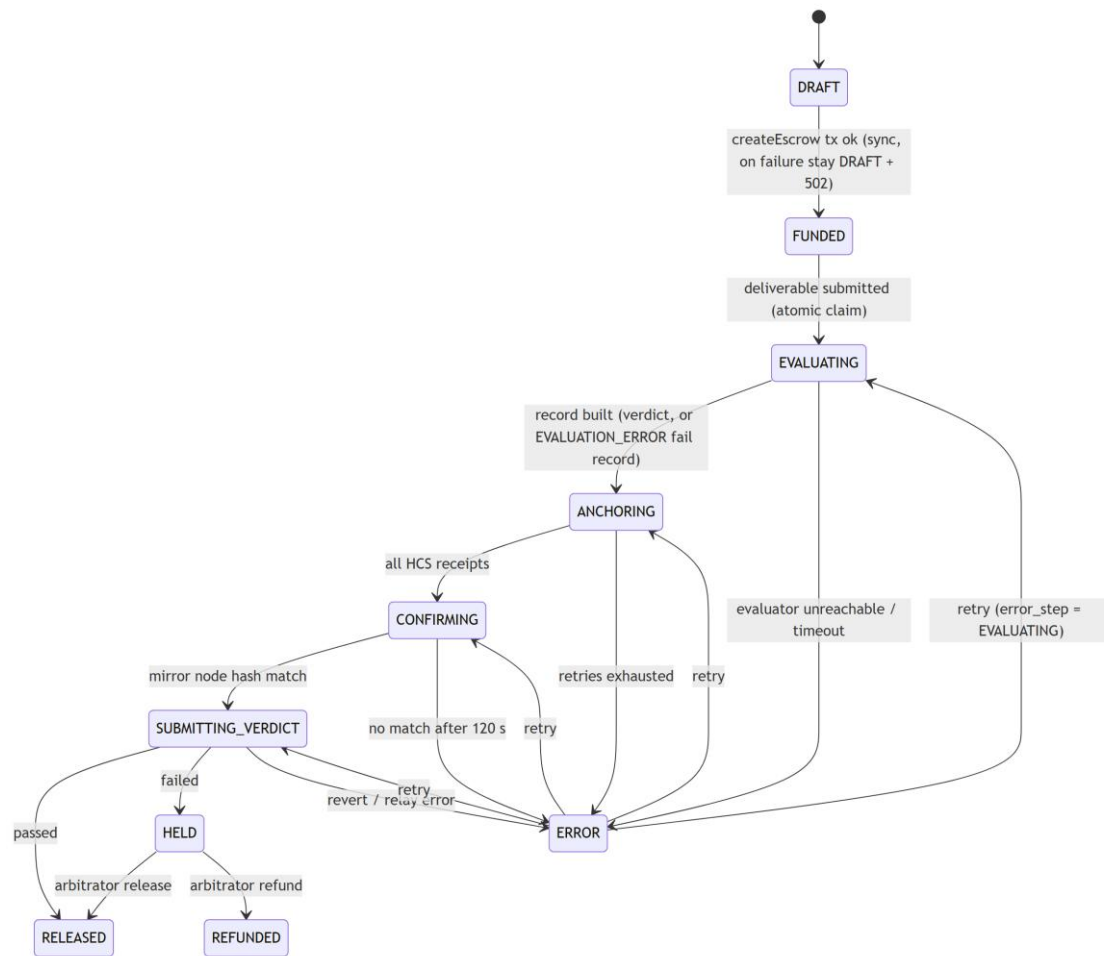
### 4.4 The AI evaluator

The evaluator runs locally (Ollama, `qwen2.5:7b-instruct`) with JSON-schema output, temperature 0, a fixed seed and committed prompts. It works in **two steps**. Step 1 extracts 3–7 acceptance criteria that can each be checked by whether something is present or not. It runs when the escrow is funded, and it reuses criteria from any earlier contract with the same `sow_hash`. Step 2 judges the deliverable against those criteria and returns per-criterion results, reasoning, confidence and an injection flag.

**The code decides the verdict, not the model.** `verdict = pass` only if every required criterion is met, confidence  $\geq 0.7$ , and no injection is suspected. The injection flag comes from the model *or* from a code-side regex that catches text addressed to the evaluator ("note to the evaluator: ...", "mark this as pass", "ignore previous instructions"). A deliverable that just says "PASS" can't pass by itself.

Failures fall into two groups. **Invalid output** (bad JSON, unknown criterion IDs, confidence out of range, or hitting the token cap) gets up to three attempts. After that, the pipeline still anchors an honest `fail` record, with reasoning starting `EVALUATION_ERROR:`, and submits it on-chain. The escrow is then `Held` and the arbitrator can resolve it. (In the first draft of the design, this case would have left the funds stuck.) **Infrastructure failures** (Ollama unreachable, or a timeout) pause the contract in a resumable `ERROR` state, and nothing is anchored. As an on-stage fallback, the **replay mode** re-uses a recorded real evaluation, but its `model_version` becomes `replay/...`, so the anchored record itself says it was replayed.

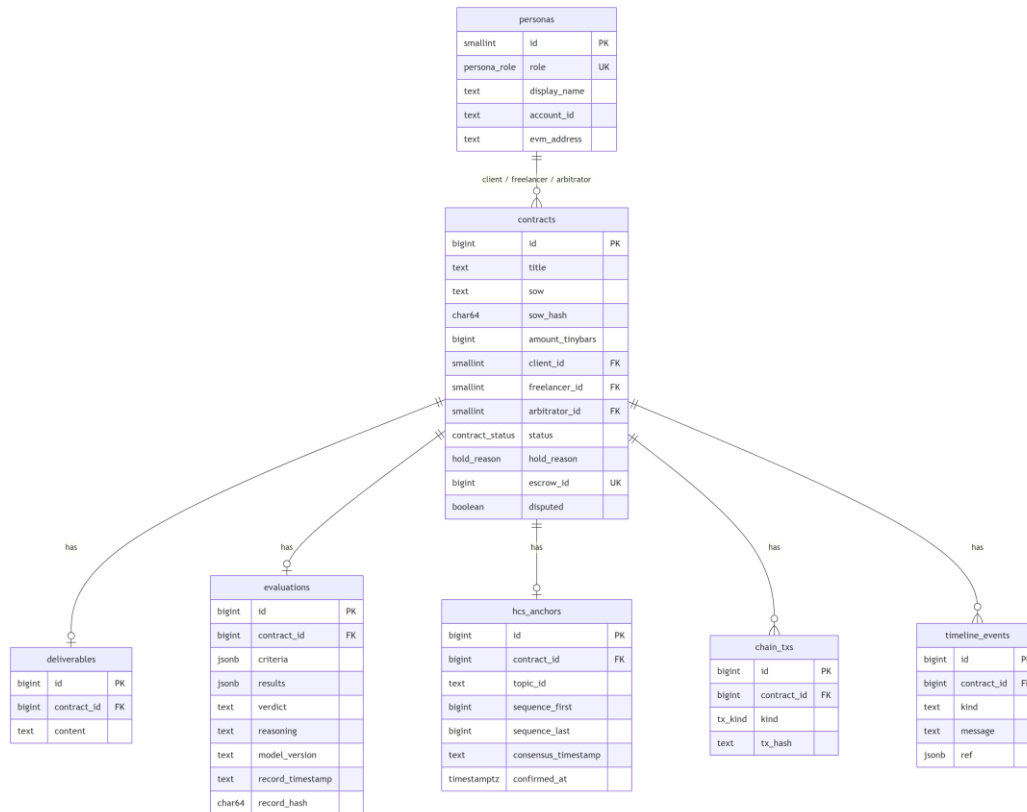
## 4.5 Orchestration: a resumable state machine



**Figure 4.** Evaluation state machine. ERROR is a paused state: Retry resumes from the step that failed.

The pipeline status lives on `contracts.status`, and every step is idempotent. `POST /deliverable` claims the work with a conditional `UPDATE ... WHERE status='FUNDED'` instead of a row lock. **No database transaction stays open during an LLM or network call**, which also keeps the demo's `psql` attack from blocking. On restart, `api` resumes every in-flight contract with rules chosen so nothing is sent twice. `CONFIRMING` only polls again and never resubmits. `SUBMITTING_VERDICT` first reads the escrow on-chain and, if the verdict has already landed, updates the database from the chain instead of calling the contract again. Both the crash-resume and the retry paths were exercised on testnet during the Day 2 gate.

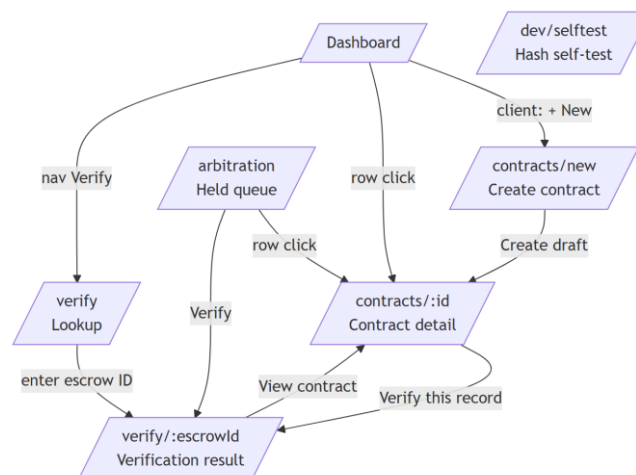
## 4.6 Data model



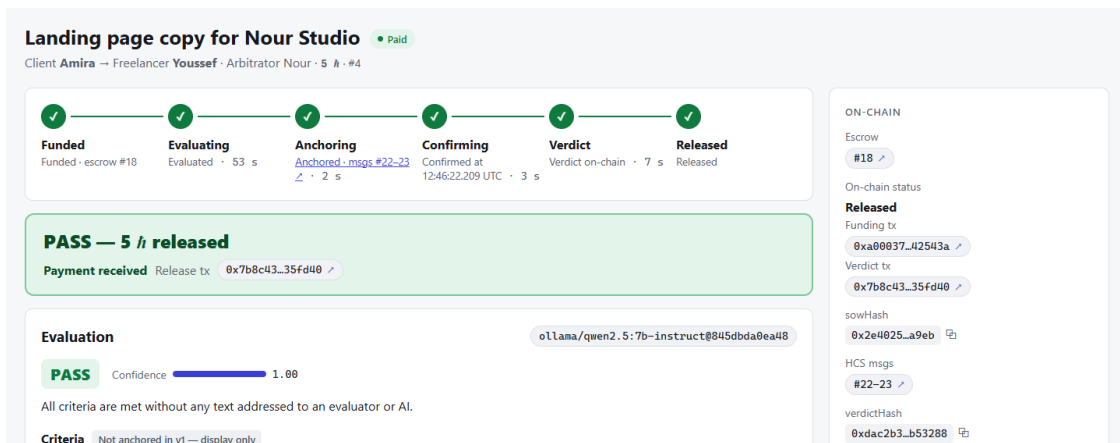
**Figure 5.** Entity-relationship diagram (Postgres 16). The view `v_canonical_record` assembles the eight hashed fields; CHECK constraints enforce address case, timestamp format and status consistency.

## 5. User experience and public verification

The web app has a persona switcher (Client, Freelancer, Arbitrator) in the header and one route per job. The contract detail page is where the demo happens. A live stepper runs *Evaluating* → *Anchoring to HCS* → *Confirming consensus* → *Submitting verdict* → *Paid/Held*, and every step has a HashScan link.



**Figure 6.** Screen map.



**Figure 7.** Live run on testnet (escrow #18): stepper with per-step timings, the PASS card and the on-chain panel with funding and verdict transactions, sowHash, HCS message numbers and verdictHash.

### The verification algorithm (runs entirely in the browser)

1. Fetch the app's copy of the record from `api/verify/{escrowId}`, keyed by the **on-chain** escrow ID.
2. Fetch the HCS chunks from the mirror node using the **bundled** topic ID, reassemble them, and hash them with `@noble/hashes`.
3. Check that the anchored record's `contract_id` equals the requested escrow. This stops the backend from pairing an honest record with the wrong escrow.
4. Hash the app's copy. **MATCH** if the two hashes are equal; otherwise **TAMPERING DETECTED** with a field-by-field diff. The anchored record's own verdict and reasoning are always shown as the authoritative version.
5. **Oracle-consistency check:** read `escrows(id)` through the mirror node's `contracts/call` (no keys, no signing) and check four things: `verdictHash` equals the anchored hash, `verdictPassed` equals the anchored verdict, `sowHash` equals the hash of the anchored SOW, and the escrow exists.
6. **Topic scan:** read the whole topic without backend pointers. It still finds the record after the database rows are deleted, and it raises a red flag if two *different* records exist for one escrow.



**Figure 8.** How a database edit is caught: the displayed hash no longer equals the anchored hash, while the contract still commits to the anchored record.

PUBLIC VERIFICATION

**Escrow #16** Re-run checks

TAMPERING DETECTED — the record shown by this app does not match the record anchored on Hedera.

- ✓ Loaded displayed record from app database
- ✓ Fetched HCS messages #18-19 (2 chunks) from testnet mirror node
- ✓ Anchored record belongs to escrow #16
- ✓ Hashed displayed record in your browser → b0f3..a23f
- ✓ Hashed HCS record in your browser → 03f3..ed17
- ✓ Escrow contract commits to the same record

Displayed hash b0f3..a23f Anchored hash 03f3..ed17

**Field-by-field comparison**

| FIELD                                              | SHOWN BY APP (DB)                                                  | ANCHORED ON HEDERA                                                                                                    |
|----------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| contract_id                                        | (identical)                                                        |                                                                                                                       |
| deliverable                                        | (identical)                                                        |                                                                                                                       |
| model_version                                      | (identical)                                                        |                                                                                                                       |
| reasoning <span style="color: red;">changed</span> | All acceptance criteria are met. The FAQ is complete and accurate. | The deliverable meets all criteria except for C2, which does not state the prices of the 1-litre and 3-litre bottles. |
| schema                                             | (identical)                                                        |                                                                                                                       |
| sow                                                | (identical)                                                        |                                                                                                                       |
| timestamp                                          | (identical)                                                        |                                                                                                                       |
| verdict <span style="color: red;">changed</span>   | pass                                                               | fail                                                                                                                  |

**The anchored version is the one produced at evaluation time. Judge that one.**

**EVIDENCE**

**Escrow contract** ✓ / **HCS** ✓ / **App** ✗

- ✓ verdictHash = anchored record hash
- ✓ verdictPassed = anchored verdict
- ✓ sowHash = hash of anchored SOW
- ✓ escrow exists on-chain

Topic (built into this page)

0.0.10748183 ✓

HCS messages

#18-19 - 2 chunks ✓

Consensus timestamp

2026-09-29 12:37:10.778 UTC

Escrow

#16 = 0x3dc2..f816 ✓

On-chain status

**Held**

Topic scan (no backend pointers)

1 distinct record for escrow #16

On-chain verdictHash

0x03f3373a..03ed17 🔗

**Anchored record** (from Hedera) View contract

Verdict FAIL Model o1Lama/qwen2.5:7b-instruct@845dbda9ea48 Timestamp 2026-09-29T12:37:09.164Z

**Reasoning**

The deliverable meets all criteria except for C2, which does not state the prices of the 1-litre and 3-litre bottles.

- Statement of Work
- Deliverable

**Figure 9.** The demo attack, live: after `\i /demo/tamper.sql` flips S2's verdict to pass in Postgres, `/verify/16` turns red, highlights verdict and reasoning in the diff, keeps showing the anchored FAIL from Hedera, and reports "Escrow contract ✓ / HCS ✓ / App ✗".

## 6. Security and trust model

Our aim is to claim exactly what the system enforces, and nothing more. Figure 10 and Table 4 separate what is proven, what is mitigated and what is not proven.

**Relation to HIP-478.** Hedera contracts can't read HCS directly today. HIP-478 (status *Accepted*) proposes that contracts and HCS interact through an oracle network. It lists direct topic reads under *Rejected Ideas*: possible, but rejected in favour of the oracle route for consistency. Our design is that oracle pattern. What we add is that **the oracle can't lie quietly**, because every claim it makes can be checked in any browser.

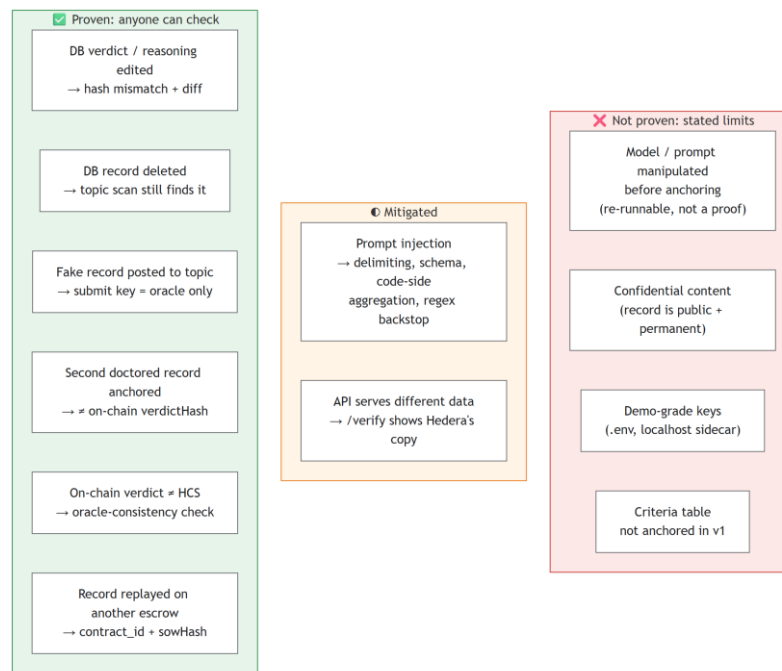


Figure 10. Trust model: proven, mitigated, not proven.

Table 4. Threat model summary (TRD §15).

| Threat                                       | Status       | How it is handled                                                                                                                                    |
|----------------------------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Operator edits verdict/reasoning in the DB   | Caught       | Hash mismatch shown publicly; field diff (the demo)                                                                                                  |
| Operator deletes the DB rows                 | Caught       | Full record lives on HCS; topic scan still finds it                                                                                                  |
| Third party posts a fake record              | Prevented    | Topic submit key = oracle only                                                                                                                       |
| Operator anchors a second, doctored record   | Caught       | On-chain <b>verdictHash</b> differs; topic scan shows multiple records                                                                               |
| Oracle's on-chain verdict differs from HCS   | Caught       | <b>verdictHash</b> + <b>verdictPassed</b> checked in the browser                                                                                     |
| Record replayed across escrows / deployments | Prevented    | <b>contract_id</b> in the record; one topic per deployment, bundled in the build                                                                     |
| Prompt injection in the deliverable          | Mitigated    | Delimiting, schema output, code aggregation, model flag + regex                                                                                      |
| Manipulation <i>before</i> anchoring         | Not proven   | Inputs, model version and prompts are public, so anyone can re-run the evaluation; LLM output isn't bit-reproducible, so this is evidence, not proof |
| Confidential content                         | Out of scope | Full record is public and permanent; production would anchor encrypted payloads                                                                      |

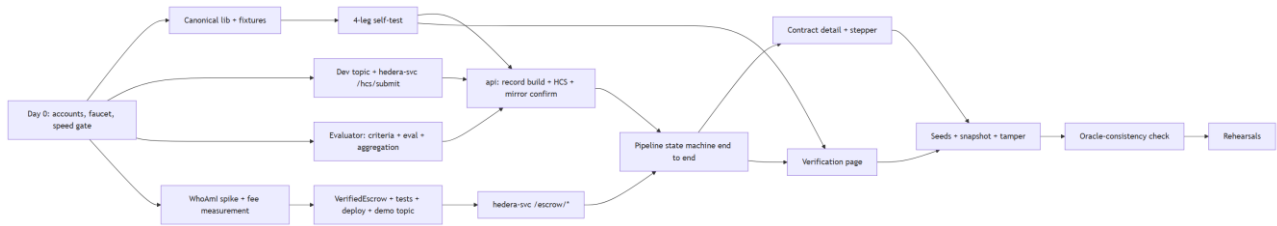
## 7. Development method

This was a solo build over one week, so the plan was to write the specifications first and treat risk as a series of gates. Before any code, a spec (v2) was expanded into seven owned documents: PRD, TRD, App Flow, Backend Schema, Implementation Plan, Demo Content, and a review report. Each topic has exactly one owning document.

The **v1.1 design review** found **47 issues**, **5 of them critical**, and wherever possible each claim was *tested rather than argued*. The contract was compiled and its 8 tests run. The canonical JSON was run on every control character in both languages. The DDL was loaded into Postgres 16 with bad-data probes. The HCS SDK source was read to confirm chunking behaviour. The critical findings would have broken the demo or made a public claim false:

- An evaluation error left escrowed funds stuck, because the arbitrator's resolve required an on-chain **HeId** status that was never set.
- The HIP-478 line in the pitch misquoted the proposal.

- "The oracle is publicly checkable" wasn't actually checked: a second, doctored record would have shown green.
- The verify page took its topic ID from the backend it was supposed to check.
- The verdict could be revealed before it was anchored, and three documents disagreed about when.



**Figure 11.** Critical path. The four risks that could sink the project (hash drift, Hedera EVM quirks, model quality, model speed and HBAR supply) were cleared by Day 1.

**Table 5.** Daily gates and outcomes (progress reports, Days 1–4).

| Day            | Gate                                                              | Outcome                                                                                 |
|----------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Day 0 (26 Sep) | Accounts, faucet, model speed $\leq 40$ s                         | 7B too slow for both steps (65–84 s) → fallback 1: criteria extracted at funding (29 s) |
| Day 1 (27 Sep) | Four-way hash check; real verdict anchored and mirror-confirmed   | Passed first attempt; TS hash of mirror bytes = Python hash                             |
| Day 2 (27 Sep) | Pass, fail and forced-error paths end to end on testnet over HTTP | Passed; crash-resume and retry exercised; 4 Hashio relay fixes                          |
| Day 3 (29 Sep) | Full run sheet in the browser incl. tamper → red                  | Passed first attempt on the production build                                            |
| Day 4 (29 Sep) | P1 features, replay fallback, projector polish, README            | Feature-complete; tagged <b>v0.9-feature-complete</b>                                   |

## 8. Testing and results

**Table 6.** Test evidence.

| Area              | Evidence                                                                                           | Result                                                              |
|-------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Canonical hashing | 6 fixtures × (pytest, vitest, browser self-test, <b>sha256sum</b> )                                | 4/4 legs agree; self-test PASS (6/6)                                |
| Backend           | <b>pytest api/tests</b> (canonical, aggregation, criteria-once, mirror reassembly, replay)         | 44/44 passing                                                       |
| Contract          | Hardhat: release, hold, arbitration, access control, SOW mismatch, double submit, distinct parties | 8/8 passing                                                         |
| Evaluator quality | 6 SOW/deliverable pairs × 3 runs, both steps, shipped prompt                                       | <b>18/18 correct</b> ; S3 injection flagged every run; no flips     |
| Verification      | Oracle check on S1–S3; tamper; deleted rows; injected duplicate records                            | MATCH 4/4 → App X after tamper → FOUND_ON_HEDERA → MULTIPLE_RECORDS |
| UI                | Playwright scripts for the run sheet, toasts, offline bar, verify states                           | All pass on the production build                                    |

**Table 7.** Performance and cost.

| Measurement (demo laptop, GTX 1650 Ti 4 GB, 15 GB RAM) | Value                                                                                                                            |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Deliverable → Paid, live model, 3 production runs      | <b>47–53 s</b> (evaluation 33–35 s; anchoring 0–4 s; mirror confirmation 2–5 s; verdict tx 7–10 s). Requirement $\leq 60$ s: met |
| Deliverable → Paid, replay fallback (model unloaded)   | <b>11.6 s</b>                                                                                                                    |
| Evaluator suite case, both steps                       | 45–62 s                                                                                                                          |
| Fee per contract call ( <b>gasLimit</b> 400,000)       | 0.0246 $\hbar$ ; contract deploy 0.107 $\hbar$                                                                                   |

| Measurement (demo laptop, GTX 1650 Ti 4 GB, 15 GB RAM) | Value                                         |
|--------------------------------------------------------|-----------------------------------------------|
| Canonical record size                                  | 1.7–4.6 KB (2–5 HCS chunks); hard limit 18 KB |

We also tried one optimisation and dropped it because of the data. Shortening the model's output gained no measurable time, and it flipped the S1 verdict on the second suite run. The shipped prompt was then re-run three times with no flips, so it stays. A  $\leq 40$  s stage target was set as a stretch goal and missed. Rather than switch models late, we accepted about 50 s, which is within the 60 s requirement, and budgeted it in the run sheet.

**Live on testnet:** contract `0x3dc2bb3a...a4f016`, record topic `0.0.10748103`, oracle `0.0.10742743`. Three seeded escrows ran the real pipeline with 5  $\hbar$  each: #15 S1 PASS → Released; #16 S2 FAIL (missing prices) → Held; #17 S3 FAIL with the prompt injection flagged → Held. All funding, HCS and verdict transactions are linked from the README and resolve on HashScan.

## 9. Limitations and future work

- **No deadlines:** if the oracle never submits a verdict, the funds stay **Funded**. A timeout refund is the first contract change for production.
- **Before anchoring:** we can't prove the operator ran the model honestly. Future work: add a **prompt\_version** hash to the record, and look at attested or multi-party evaluation.
- **Criteria aren't anchored in v1:** the per-criterion table is labelled "not anchored" in the UI. A **detail** key would bring it under the hash.
- **Public content:** production needs encrypted payloads, or hashes with off-chain storage.
- **Demo-grade custody:** personas are backend-managed keys. Real deployments need wallet connection. An HTS "Verified Delivery" receipt NFT is a planned P2.

## 10. Conclusion

Verified-Then-Paid Escrow shows that an AI can be allowed to move money without asking anyone to trust it blindly. Hedera provides the three pieces that make this work: HCS for ordered, timestamped anchoring that the oracle alone can write; a smart contract that commits to the same hash it pays out on; and a public mirror node that lets any browser re-derive the whole chain of evidence. The result runs end to end on testnet and has been measured and rehearsed. It also states its own limits. On stage, a live database attack is caught in seconds, and the anchored record still says what the AI actually decided.